

形式语言与自动机

Formal Languages and Automata Theory

上下文无关文法

计算机科学与技术学院
哈尔滨工业大学（深圳）



郑宜峰

- **上下文无关文法**
 - 形式定义
 - 归约和派生
 - 最左派生和最右派生
 - 文法的语言
- **语法分析树**
- **文法和语言的歧义性**
- **文法的化简与范式**

- **上下文无关文法**
 - 形式定义
 - 归约和派生
 - 最左派生和最右派生
 - 文法的语言
- 语法分析树
- 文法和语言的歧义性
- 文法的化简与范式

- 为什么引入上下文无法文法CFG (context-free grammar)?
- 可以表示大多数程序设计语言的语法，尤其是程序中的配对或者嵌套的结构(比如括号配对)
- 在程序设计语言的设计、编译器的实现等方面有重要应用，也可应用在可扩展标记语言(XML) 的格式类型定义(DTD) 中等。

- 简化的英语语法规则：

$\langle \text{sentence} \rangle \rightarrow \langle \text{noun-phrase} \rangle \langle \text{verb-phrase} \rangle$

$\langle \text{noun-phrase} \rangle \rightarrow \langle \text{article} \rangle \langle \text{noun} \rangle \mid \langle \text{article} \rangle \langle \text{adjective} \rangle \langle \text{noun} \rangle$

$\langle \text{verb-phrase} \rangle \rightarrow \langle \text{verb} \rangle \mid \langle \text{verb} \rangle \langle \text{noun-phrase} \rangle$

$\langle \text{article} \rangle \rightarrow \text{a} \mid \text{the}$

$\langle \text{noun} \rangle \rightarrow \text{boy} \mid \text{girl} \mid \text{cat}$

$\langle \text{adjective} \rangle \rightarrow \text{big} \mid \text{small} \mid \text{blue}$

$\langle \text{verb} \rangle \rightarrow \text{sees} \mid \text{likes}$

...

- 使用定义的文法规则产生句子：

$\langle sentence \rangle \Rightarrow \langle noun\text{-}phrase \rangle \langle verb\text{-}phrase \rangle$
 $\Rightarrow \langle article \rangle \langle noun \rangle \langle verb\text{-}phrase \rangle$
 $\Rightarrow \langle article \rangle \langle noun \rangle \langle verb \rangle \langle noun\text{-}phrase \rangle$
 $\Rightarrow \langle article \rangle \langle noun \rangle \langle verb \rangle \langle article \rangle \langle adjective \rangle \langle noun \rangle$
 $\Rightarrow \text{the } \langle noun \rangle \langle verb \rangle \langle article \rangle \langle adjective \rangle \langle noun \rangle$
 $\Rightarrow \text{the girl } \langle verb \rangle \langle article \rangle \langle adjective \rangle \langle noun \rangle$
 $\Rightarrow \dots$
 $\Rightarrow \text{the girl sees a blue cat}$

从sentence范畴到具体的一个句子，我们所使用的工具，就是文法。

- 定义: 如果字符串 $w \in \Sigma^*$ 满足 $w = w^R$, 则称字符串 w 为回文。
- 定义: 如果语言 L 的字符串都是回文, 则称 L 为回文语言。

$$L = \{w \in \Sigma^* \mid w = w^R\}$$

- 例:

$\varepsilon, 010, 0000, radar, racecar, drawkward$

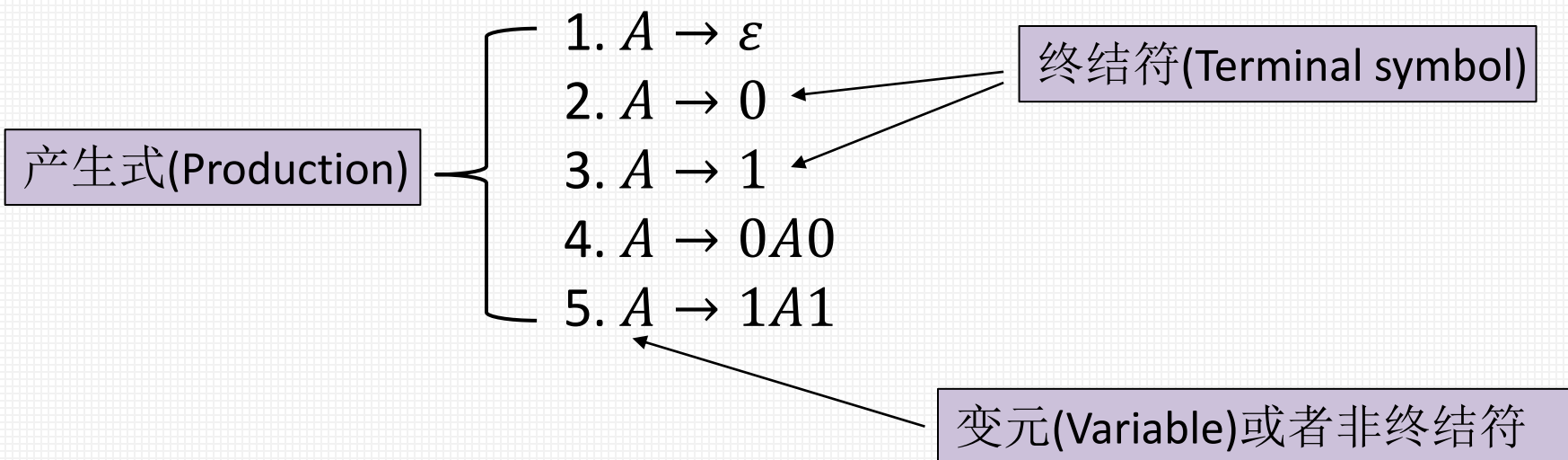
- 例：字母表 $\Sigma = \{0, 1\}$ 上的回文语言

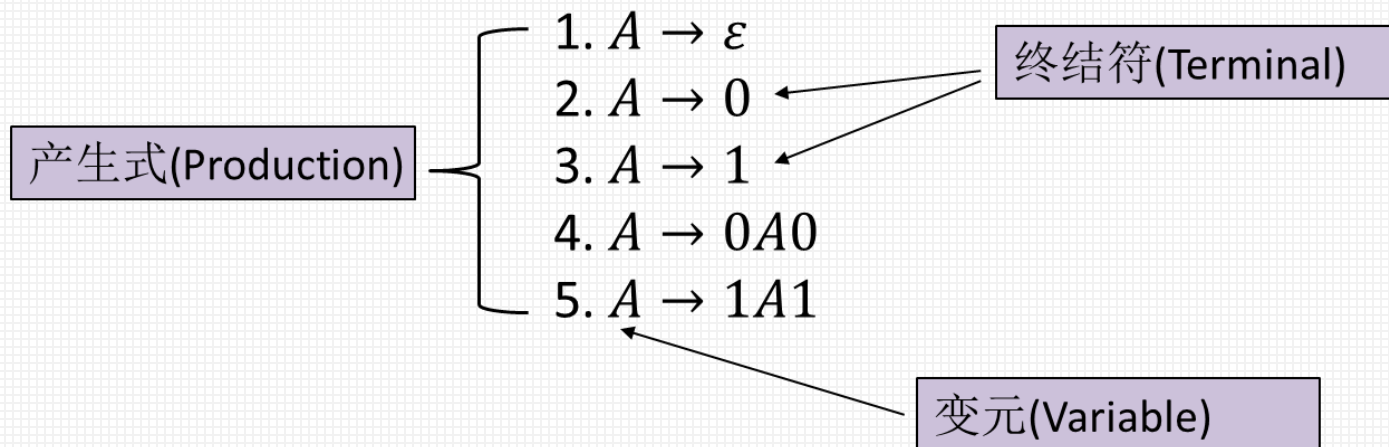
$$L_{pal} = \{w \in \{0,1\}^* \mid w = w^R\}$$

- 很容易用泵引理证明 L_{pal} 是非正则的（**? 尝试证明下，比如考虑 $w = 0^N 1 0^N$** ）。
- 因此前面的有穷自动机和正则表达式无法表示它，如何表示该语言呢？

- 例：字母表 $\Sigma = \{0, 1\}$ 上的回文语言
$$L_{pal} = \{w \in \{0,1\}^* \mid w = w^R\}$$
- 如何表示该语言呢？可使用递归的方式来定义：
 - 首先， ε , 0, 1 都是回文
 - 如果 w 是回文， $0w0$ 和 $1w1$ 也是回文
- 使用嵌套定义表示这种递归结构
 - 使用变元 A 表示回文语言中的字符串
 - $A \rightarrow \varepsilon, A \rightarrow 0, A \rightarrow 1$
 - $A \rightarrow 0A0$ （仍为回文）， $A \rightarrow 1A1$ （仍为回文）。

- 回文语言文法





- 例:使用文法的嵌套定义表示一个字符串0110

- 利用 $A \rightarrow 0A0$, 那么有 $A = 0A0$
- 利用 $A \rightarrow 1A1$, 那么有 $A = 01A10$
- 利用 $A \rightarrow \varepsilon$, 那么有 $A = 01\varepsilon10 = 0110$

过程:

1. 选取能够产生符号串的一系列产生式
2. 在每一步, 用某个产生式替换某个变元

- 定义:上下文无关文法(CFG, Context-Free Grammar, 简称文法)
 G 是一个四元组

$$G = (V, T, P, S)$$

- 其中,
 - V :变元的有穷集, 变元也称为非终结符或语法范畴
 - T :终结符的有穷集, 且 $V \cap T = \emptyset$ (即字母表), 即语言里面的符号
 - P :产生式的有穷集, 每个产生式包括:
 - 一个变元, 称为产生式的头(head)或左部;
 - 一个产生式符号 $\rightarrow$, 读作定义为;
 - 一个 $(V \cup T)^*$ 中的符号串, 称为体(body)或右部 (即由变元和终结符组成的字符串, 或者是空串)
 - S :初始符号(start symbol) $S \in V$, 表示文法开始的地方 (即如果文法有多个变元, 需要指定一个作为初始符号)

- 产生式 $A \rightarrow \alpha$, 读作 A 定义为 α
- 如果有多个 A 的产生式:

$$A \rightarrow \alpha_1, A \rightarrow \alpha_2, \dots, A \rightarrow \alpha_n$$

- 可简写为:

$$A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$$

- 文法中变元 A 的全体产生式, 称为 A 产生式
- 续例: 回文语言的文法可以表示为

$$G = (\{A\}, \{0,1\}, \{A \rightarrow \varepsilon | 0 | 1 | 0A0 | 1A1\}, A)$$

- 变元: $A, B, C, \dots$
- 终结符: $0, 1, a, b, c, \dots$
- 终结符或非终结符: $\dots, X, Y, Z$
- 终结符构成的串: $\dots, w, x, y, z$
- 终结符或非终结符组成的串: $\alpha, \beta, \gamma, \dots$

- 1. CFG: $G = (\{S\}, \{0,1\}, P, S), P: S \rightarrow 0S \mid S1 \mid \varepsilon$
 - 语言 $L = \{0^*1^*\}$

- 2. CFG: $G = (\{S\}, \{0,1\}, P, S)$, $P: S \rightarrow \varepsilon \mid 0S1$
 - 语言 $L = \{0^n 1^n \mid n \geq 0\}$
 - 注意, 若 $n > 0$, 则 $P: S \rightarrow 01 \mid 0S1$

- 3. CFG: $G = (\{S\}, \{0,1\}, P, S)$, $P: S \rightarrow \varepsilon \mid 0S11$
 - 语言 $L = \{0^n 1^{2n} \mid n \geq 0\}$

- 4. CFG: $G = (\{S, A\}, \{0, 1\}, P, S)$, $P: S \rightarrow A10, A \rightarrow 0A1 \mid \varepsilon$
 - 语言 $L = \{0^n 1^{n+1} 0 \mid n \geq 0\}$

- 两个变元：

- 用于表示表达式(expressions)本身（表达式之间可以进行“加”和“乘”运算，以及括号运算）
- 用于表示标识符(identifiers):以 $\{a, b\}$ 开头,由 $\{a, b, 0, 1\}$ 组成的字符串

- 用文法 G_{exp} 表示, $G_{exp} = (\{E, I\}, \{a, b, 0, 1, +, *, (,)\}, P, E)$, 其中产生式集 P 中有10条产生式。

$$1. E \rightarrow I$$

$$2. E \rightarrow E + E$$

$$3. E \rightarrow E * E$$

$$4. E \rightarrow (E)$$

$$5. I \rightarrow a$$

$$6. I \rightarrow b$$

$$7. I \rightarrow Ia$$

$$8. I \rightarrow Ib$$

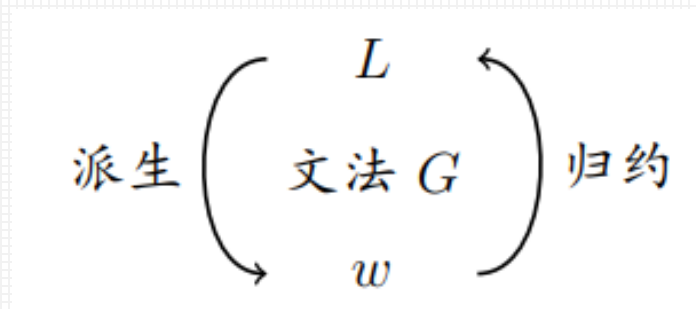
$$9. I \rightarrow I0$$

$$10. I \rightarrow I1$$

- 上下文无关文法
 - 形式定义
 - 归约和派生
 - 最左派生和最右派生
 - 文法的语言
- 语法分析树
- 文法和语言的歧义性
- 文法的化简与范式

- 非形式定义：

- 从字符串到文法变元的分析过程, 称为归约(reduction);
- 从文法变元到字符串的分析过程, 称为派生(**derivation**)。



- 归约：自底向上，由产生式的体向头的分析
- 派生：自顶向下，由产生式的头向体分析

- 用算数表达式文法 G_{exp} , 将 $a * (a + b00)$ 归约的过程

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$
5. $I \rightarrow a$
6. $I \rightarrow b$
7. $I \rightarrow Ia$
8. $I \rightarrow Ib$
9. $I \rightarrow I0$
10. $I \rightarrow I1$

$a * (a + b00)$:

利用 (5), (6) : $I * (I + I00)$

利用 (9) : $I * (I + I0)$

利用 (9) : $I * (I + I)$

利用 (1) : $E * (E + E)$

利用 (2) : $E * E$

利用 (3) : E

可以归约到文法的开始符号 E 。

- 定义: 若上下文无关文法 $G = (V, T, P, S)$, 设 $\alpha, \beta, \gamma \in (V \cup T)^*$, $A \in V, A \rightarrow \gamma \in P$, 那么称在 G 中由 $\alpha A \beta$ 可派生出 $\alpha \gamma \beta$, 记为

$$\alpha A \beta \Rightarrow_G \alpha \gamma \beta.$$

- 相应的, 称 $\alpha \gamma \beta$ 可归约为 $\alpha A \beta$.
- 派生 $\alpha A \beta \Rightarrow_G \alpha \gamma \beta$, 即是用产生式 $A \rightarrow \gamma$ 的右部 γ 替换串 $\alpha A \beta$ 中变元 A 得到串 $\alpha \gamma \beta$ 的过程。
- 如果语境中 G 是已知的, 可省略, 记为 $\alpha A \beta \Rightarrow \alpha \gamma \beta$

- 设 $\alpha_1, \dots, \alpha_m \in (V \cup T)^*$, $m \geq 1$, 对 $i = 1, \dots, m - 1$, 如果有

$$\alpha_i \xRightarrow{G} \alpha_{i+1}$$

成立, 即 α_1 通过零步或者多步派生可得到 α_m

$$\alpha_1 \xRightarrow{G} \alpha_2 \xRightarrow{G} \dots \xRightarrow{G} \alpha_m$$

那么记为 $\alpha_1 \xRightarrow{*}_G \alpha_m$ 。

- 若 α 派生出 β 刚好经过了 i 步, 可记为 $\alpha \xRightarrow{i}_G \beta$ 。

- 例：算数表达式 $a * (a + b00)$ 在文法 G_{exp} 中的派生过程.

$$1. E \rightarrow I$$

$$2. E \rightarrow E + E$$

$$3. E \rightarrow E * E$$

$$4. E \rightarrow (E)$$

$$5. I \rightarrow a$$

$$6. I \rightarrow b$$

$$7. I \rightarrow Ia$$

$$8. I \rightarrow Ib$$

$$9. I \rightarrow I0$$

$$10. I \rightarrow I1$$

$$E \Rightarrow E * E$$

$$\Rightarrow E * (E)$$

$$\Rightarrow I * (E)$$

$$\Rightarrow I * (E + E)$$

$$\Rightarrow I * (E + I)$$

$$\Rightarrow I * (I + I)$$

$$\Rightarrow I * (a + I)$$

$$\Rightarrow a * (a + I)$$

$$\Rightarrow a * (a + I0)$$

$$\Rightarrow a * (a + I00)$$

$$\Rightarrow a * (a + b00)$$

- 上下文无关文法
 - 形式定义
 - 归约和派生
 - 最左派生和最右派生
 - 文法的语言
- 语法分析树
- 文法和语言的歧义性
- 文法的化简与范式

- 为限制派生的随意性，要求只替换符号串中最左边变元的派生过程，称为最左派生，记为

$$\Rightarrow_{lm}, \overset{*}{\Rightarrow}_{lm}$$

只替换最右的，称为最右派生，记为

$$\Rightarrow_{rm}, \overset{*}{\Rightarrow}_{rm}$$

- 任何派生都有等价的最左派生和最右派生。

$$A \overset{*}{\Rightarrow} w \text{ 当且仅当 } A \overset{*}{\Rightarrow}_{lm} w, \text{ 当且仅当 } A \overset{*}{\Rightarrow}_{rm} w。$$

- 例：表达式 $a * (a + a)$ 在 G_{exp} 中的最左派生为：

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$
5. $I \rightarrow a$
6. $I \rightarrow b$
7. $I \rightarrow Ia$
8. $I \rightarrow Ib$
9. $I \rightarrow I0$
10. $I \rightarrow I1$

$$\begin{aligned} E &\xRightarrow{\text{lm}} E * E \\ &\xRightarrow{\text{lm}} I * E \\ &\xRightarrow{\text{lm}} a * E \\ &\xRightarrow{\text{lm}} a * (E) \\ &\xRightarrow{\text{lm}} a * (E + E) \\ &\xRightarrow{\text{lm}} a * (I + E) \\ &\xRightarrow{\text{lm}} a * (a + E) \\ &\xRightarrow{\text{lm}} a * (a + I) \\ &\xRightarrow{\text{lm}} a * (a + a) \end{aligned}$$

- 例：表达式 $a * (a + a)$ 在 G_{exp} 中的最右派生为：

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$
5. $I \rightarrow a$
6. $I \rightarrow b$
7. $I \rightarrow Ia$
8. $I \rightarrow Ib$
9. $I \rightarrow I0$
10. $I \rightarrow I1$

$$\begin{aligned} E &\xRightarrow{\text{rm}} E * E \\ &\xRightarrow{\text{rm}} E * (E) \\ &\xRightarrow{\text{rm}} E * (E + E) \\ &\xRightarrow{\text{rm}} E * (E + I) \\ &\xRightarrow{\text{rm}} E * (E + a) \\ &\xRightarrow{\text{rm}} E * (I + a) \\ &\xRightarrow{\text{rm}} E * (a + a) \\ &\xRightarrow{\text{rm}} I * (a + a) \\ &\xRightarrow{\text{rm}} a * (a + a) \end{aligned}$$

- 上下文无关文法
 - 形式定义
 - 归约和派生
 - 最左派生和最右派生
 - 文法的语言
- 语法分析树
- 文法和语言的歧义性
- 文法的化简与范式

- 定义:上下文无关文法 $G = (V, T, P, S)$ 的语言定义为

$$L(G) = \{w | w \in T^*, S \xRightarrow[G]{*} w\}$$

即符号串 w 在 $L(G)$ 中要满足:

1. w 由终结符组成, 或者是空串;
2. 初始符号 S 能派生出 w .

- 如果语言 L 是某个 CFG G 定义的语言, 即 $L = L(G)$, 则称 L 为上下文无关语言 (CFL, Context-Free Language)
- **上下文无关**是指在文法派生的每一步

$$\alpha A \beta \Rightarrow \alpha \gamma \beta$$

符号串 γ 仅根据 A 的产生式派生, 而无需依赖 A 的上下文 α 和 β .

- 定义：如果有两个文法 CFG G_1 和 CFG G_2 , 若 $L(G_1) = L(G_2)$, 则称 G_1 和 G_2 是等价的.

- 若文法 $G = (V, T, P, S)$, 初始符号 S 派生出来的符号串, 称为 G 的句型 (sentential form), 即 $\alpha \in (V \cup T)^*$ 且 $S \xRightarrow{*} \alpha$
- 如果 $S \xRightarrow[lm]{*} \alpha$, 称 α 为左句型
- 如果 $S \xRightarrow[rm]{*} \alpha$, 称 α 为右句型

- 例：描述CFG $G = (\{S\}, \{a, b\}, \{S \rightarrow aSb, S \rightarrow ab\}, S)$ 所定义的语言？

- 例：描述CFG $G = (\{S\}, \{a, b\}, \{S \rightarrow aSb, S \rightarrow ab\}, S)$ 所定义的语言？

观察开始符号 S 可以派生出的串：

$$S \Rightarrow aSb \Rightarrow a^2Sb^2 \Rightarrow \dots \Rightarrow a^{n-1}Sb^{n-1}$$

利用第二条产生式，有：

$$a^n b^n$$

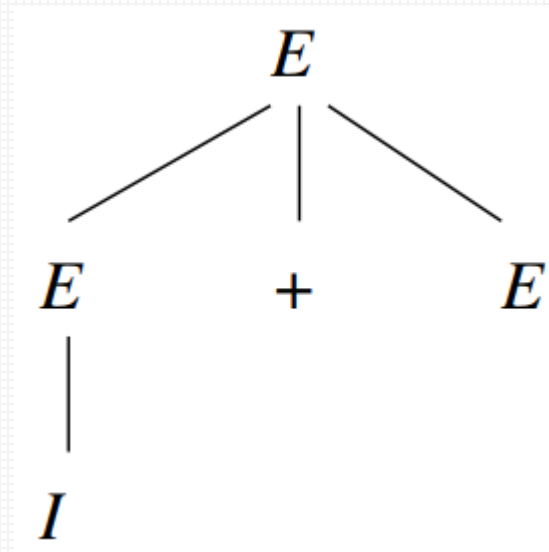
故 $L(G) = \{a^n b^n | n \geq 1\}$

- 例：描述CFG $G = (\{A\}, \{ (,) \}, \{ A \rightarrow AA \mid (A) \mid \varepsilon \}, A)$ 所定义的语言？

- 例：描述CFG $G = (\{A\}, \{(\,)\}, \{A \rightarrow AA \mid (A) \mid \varepsilon\}, A)$ 所定义的语言？
- $A \Rightarrow (A) \Rightarrow ((A)) \Rightarrow (((A))) \Rightarrow (((\,)))$ ，产生嵌套的匹配括号对
- $A \Rightarrow AA \Rightarrow (A)(A) \Rightarrow ()(A) \Rightarrow ()()$ ，产生匹配括号对的级联 (concatenation)
- 因此，该CFG能产生含匹配括号对的符号串，比如 $()$, $((\,))$, $((\,))(\,)$

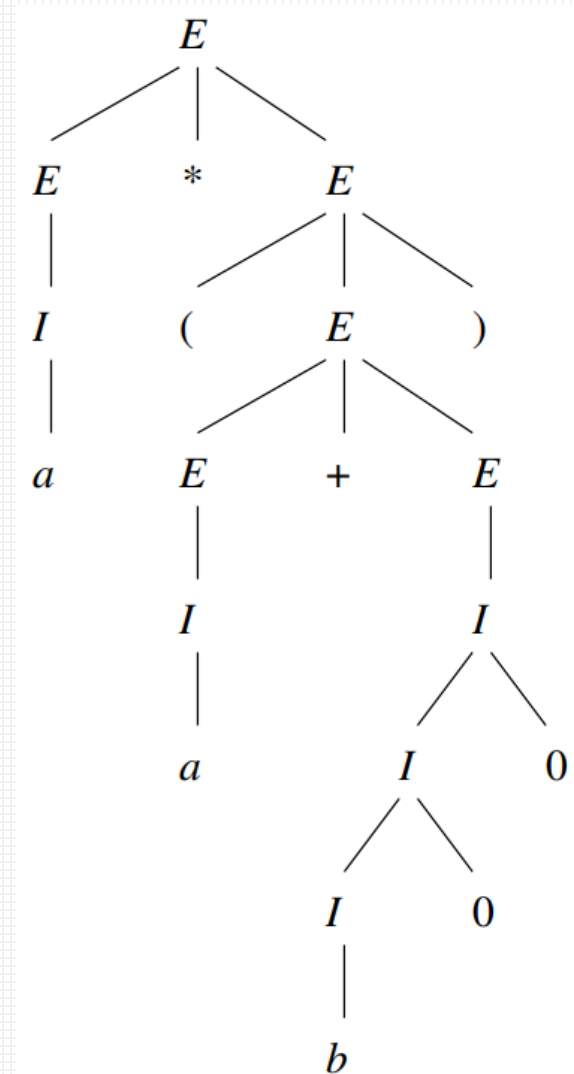
- 上下文无关文法
- **语法分析树**
 - 形式定义
 - 语法树和派生的等价性
- 文法和语言的歧义性
- 文法的化简与范式

- 语法分析树用来表示派生，可以从树中看出整个派生过程和最终产生的符号串。
- 例：对于前面算术表达式的文法，派生 $E \Rightarrow E + E \Rightarrow I + E$ 的语法分析树为



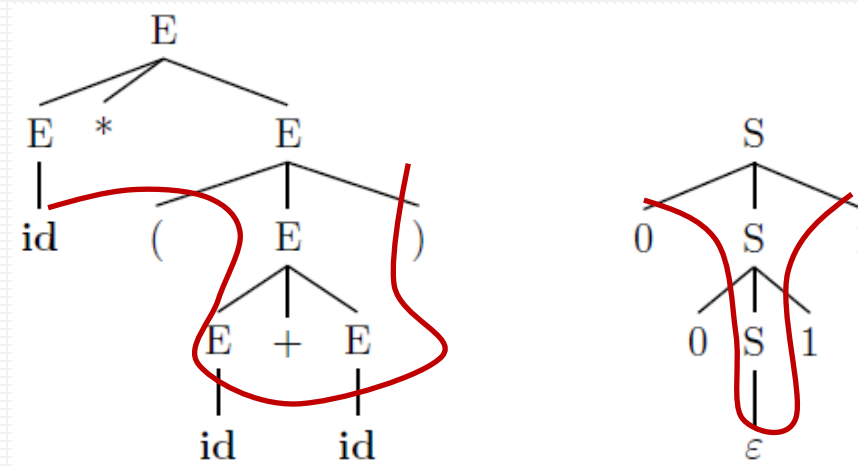
- **定义: CFG $G = (V, T, P, S)$ 的语法分析树 (parse tree), 也称为派生树 (derivation tree), 其形式定义为:**
 - 每个内节点的标记是 V 中的变元符号
 - 每个叶子节点的标记是 $V \cup T \cup \{\varepsilon\}$ 中的符号
 - 如果内节点的标记是 A , 其子节点从左至右分别为 $X_1, X_2, \dots, X_n$
- 那么, $A \rightarrow X_1 X_2 \dots X_n$ 是 P 的一个产生式;
- 若某个 X_i 为 ε , 则 X_i 一定是 A 唯一的子节点, 且 $A \rightarrow \varepsilon$ 是一个产生式。

- 定义：语法分析树的全部叶节点从左到右连接起来，称为该树的产物（yield）或结果。如果树根节点是初始符号 S ，叶节点是终结符，那么该树的产物属于 $L(G)$ 。



产物 $a*(a+b00)$ 的语法分析树

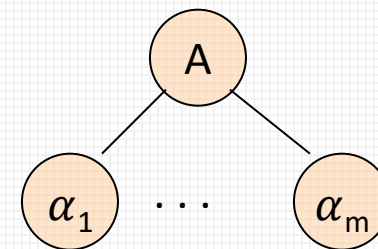
- 定义：语法分析树中标记为 A 的内节点及其全部子孙节点构成的子树，称为 A 子树。



- 定理: CFG $G = (V, T, P, S)$ 且 $A \in V$, 那么文法 G 中
$$A \xRightarrow{*} \alpha$$

当且仅当在文法 G 中存在以 A 为根节点产物为 α 的语法分析树。

- 充分性（从文法到语法分析树）：在文法 G 中若 $A \xRightarrow{*} \alpha$ ，则存在以 A 为根节点产物为 α 的语法分析树。
- 对 $A \xRightarrow{j} \alpha$ 的步骤数 j 进行归纳证明（即以 A 为根节点的树的高度）
- 归纳基础： $j = 1$ ，即 $A \Rightarrow \alpha$ ，有 $A \rightarrow \alpha$ 是产生式。



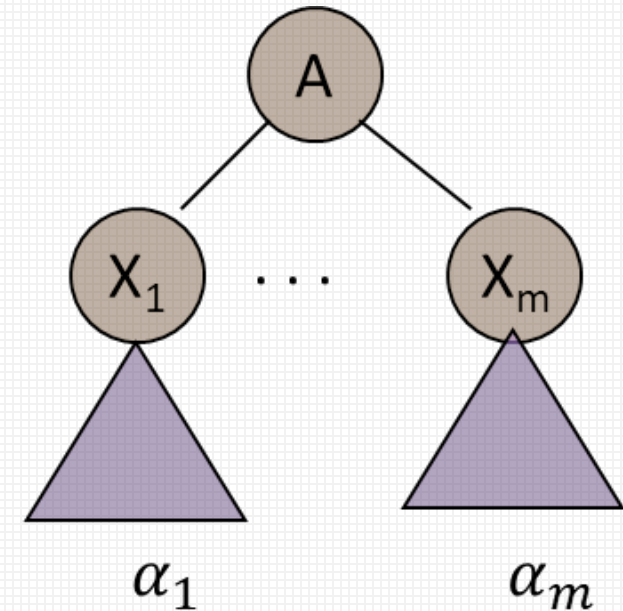
- 归纳递推：假设当 $j \leq n$ 命题成立。当 $j = n + 1$ 时， $A \xRightarrow{n+1} \alpha$ 的派生过程为：

$$A \Rightarrow X_1 X_2 \dots X_m \xRightarrow{n} \alpha_1 \alpha_2 \dots \alpha_m = \alpha$$

其中第一步派生有 $A \rightarrow X_1 X_2 \dots X_m$ 是 G 的产生式，且一定有 $X_i \xRightarrow{*} \alpha_i$ 且不超过 n 步。

由归纳假设存在一棵以 X_i 为根， α_i 为产物的一个语法分析树。

则可以构造以 A 为根，以 X_i 为子树的语法分析树，其产物刚好为 α 。



- 必要性（从语法分析树到文法）：在文法 G 中若存在以 A 为根节点产物为 α 的语法分析树，则 $A \xRightarrow{*} \alpha$ 。
- 对语法分析树的**内节点数** j 归纳证明。
- 归纳基础： $j = 1$ 时，即只有一个内节点，那么一定是根节点 A ，产物为 α ，则有 $A \rightarrow \alpha \in P$ ，那么 $A \xRightarrow{*} \alpha$ 。

- 归纳递推：假设内节点数 $j \leq n$ 时命题成立. 当 $j = n + 1$ 时，根节点子节点为 $X_1, X_2, \dots, X_m$ ，则

$$A \rightarrow X_1 \cdots X_m \in P, \text{ 且 } A \Rightarrow X_1 \cdots X_m$$

而 X_i 子树（或叶子）内节点数都不超过 n ，由归纳假设有

$$X_i \overset{*}{\Rightarrow} \alpha_i,$$

从左至右连接 α_i ，刚好为树的产物 α ，所以有

$$X_1 X_2 \cdots X_m \overset{*}{\Rightarrow} \alpha_1 X_2 \cdots X_m \overset{*}{\Rightarrow} \cdots \overset{*}{\Rightarrow} \alpha_1 \alpha_2 \cdots \alpha_m = \alpha$$

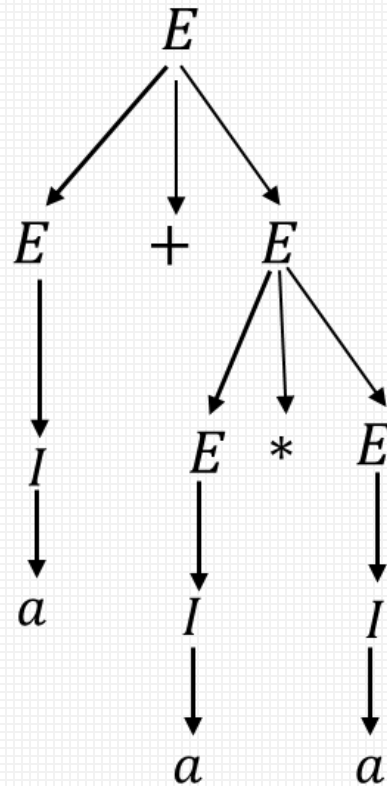
- 每棵语法分析树都有唯一的最左（右）派生
- 给定CFG $G = (V, T, P, S)$, $A \in V$, 以下命题等价:
 - 存在以 A 为根节点, 产物为 w 的语法分析树
 - $A \xRightarrow{*} w$
 - $A \xRightarrow[lm]{*} w$
 - $A \xRightarrow[rm]{*} w$

- 上下文无关文法
- 语法分析树
- **文法和语言的歧义性**
 - 文法歧义性的消除
 - 语言的固有歧义性
- 文法的化简与范式

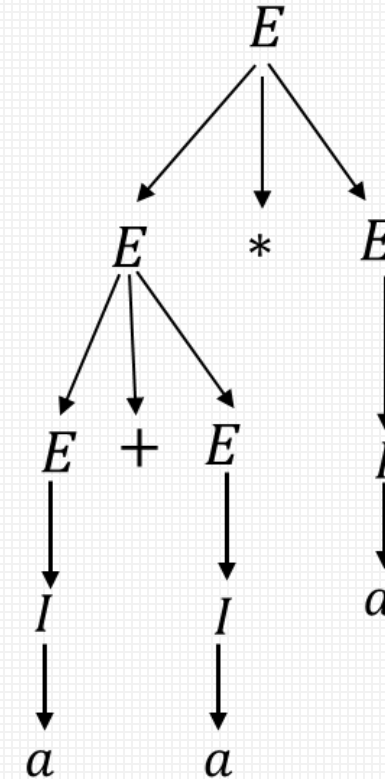
- 定义: 如果CFG G 使某些符号串有两棵不同的语法分析树, 则称该文法是歧义的 (ambiguous).

- 例. 算数表达式的文法 G_{exp} 中, 对句型 $a + a * a$ 有下面两棵语法树.

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$
5. $I \rightarrow a$
6. $I \rightarrow b$
7. $I \rightarrow Ia$
8. $I \rightarrow Ib$
9. $I \rightarrow I0$
10. $I \rightarrow I1$

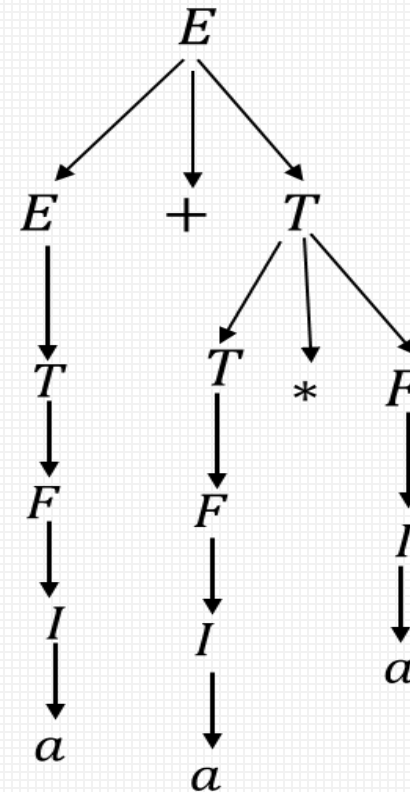


$$\begin{aligned} E &\Rightarrow E + E \\ &\Rightarrow E + E * E \\ &\Rightarrow a + a * a \end{aligned}$$



$$\begin{aligned} E &\Rightarrow E * E \\ &\Rightarrow E + E * E \\ &\Rightarrow a + a * a \end{aligned}$$

- 有些文法的歧义性，可以重新设计文法来消除
- 例续. 文法 G_{exp} 重新设计为文法 $G_{exp'}$ 可消除歧义

$$E \rightarrow E + T | T$$
$$T \rightarrow T * F | F$$
$$F \rightarrow (E) | I$$
$$I \rightarrow a$$
$$I \rightarrow b$$
$$I \rightarrow Ia$$
$$I \rightarrow Ib$$
$$I \rightarrow I0$$
$$I \rightarrow I1$$


- 定义: 定义同样的语言可以有多个文法。如果上下文无关语言 L 的所有文法都是有歧义的, 那么称语言 L 是**固有歧义**的。

- 固有歧义的语言确实存在, 如语言

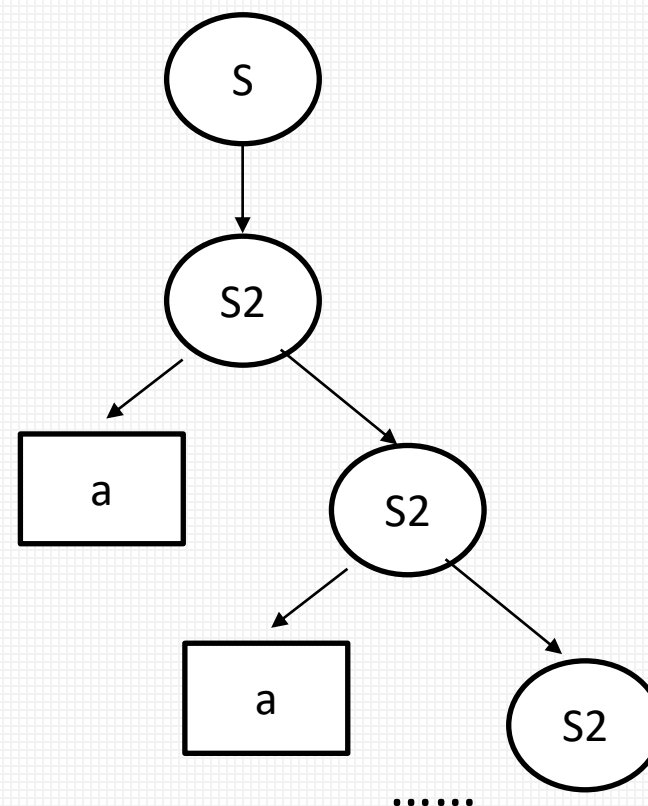
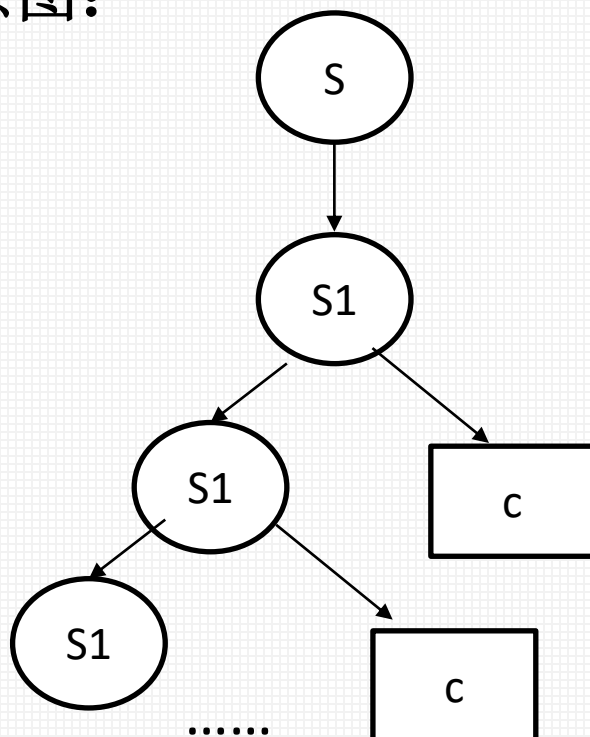
$$L = \{a^i b^j c^k \mid i = j \text{ or } j = k\}$$

是固有歧义的。

- 直观的, 至少对于其中形如 $a^n b^n c^n$ 的符号串, 语法分树可以先检查 a 和 b , 也可以是先检查 b 和 c 。

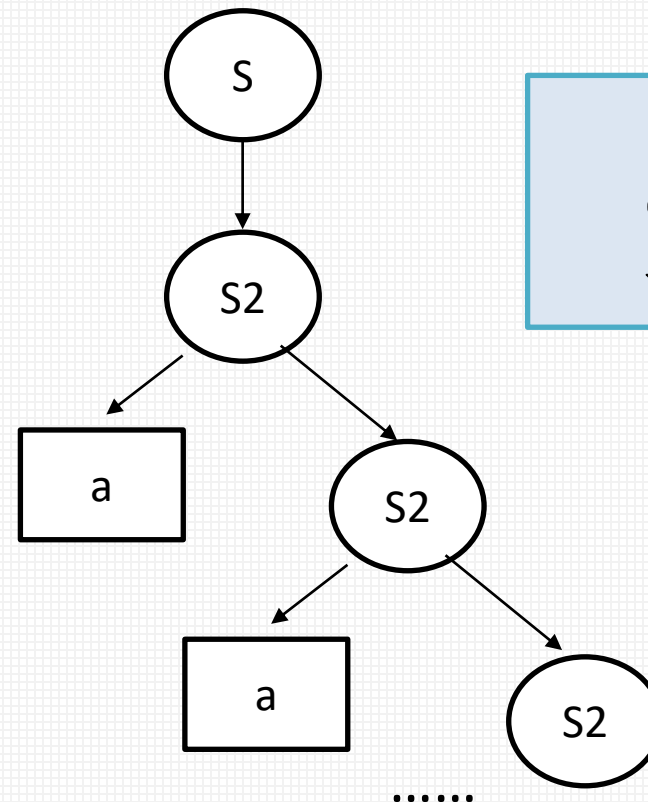
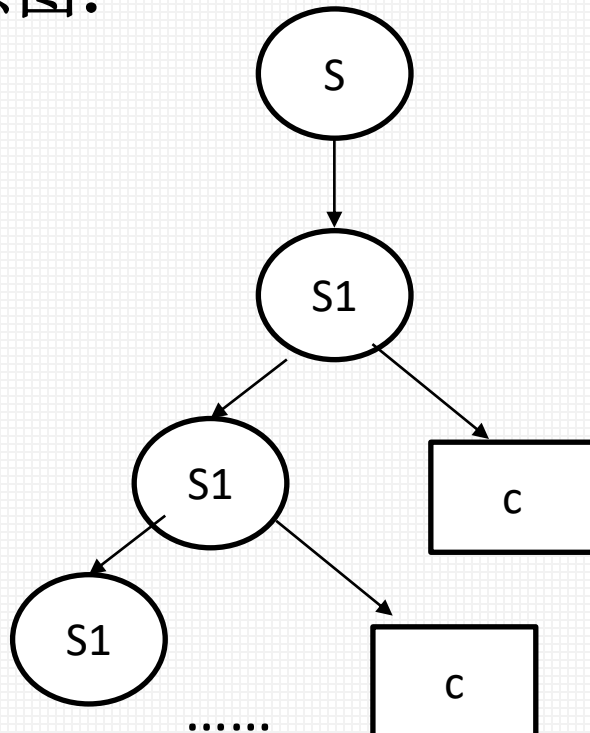
- 对于其中形如 $a^n b^n c^n$ 的符号串，语法分析树可以先检查 a 和 b ，也可以是先检查 b 和 c 。

示意图：



- 对于其中形如 $a^n b^n c^n$ 的符号串，语法分析树可以先检查 a 和 b ，也可以是先检查 b 和 c 。

示意图：



“判定任何给定CFG G 是否是歧义的”是一个不可判定问题

- 上下文无关文法
- 语法分析树
- 文法和语言的歧义性
- **文法的化简与范式**
 - 消除无用符号
 - 消除 ε – 产生式
 - 消除单元产生式
 - 乔姆斯基范式
 - 格雷巴赫范式

- 典型问题：给定CFG G 和串 w ，判断 $w \in L(G)$?
- 编译器设计和自然语言处理的基本问题之一
- 但文法的形式非常自由，过于复杂不易于自动处理
- 为了便于分析和解决问题，以不改变语言为前提，化简文法和限制文法的格式

- 消除无用符号 (useless symbols) : 对文法定义语言没有贡献的符号
- 消除 ε - 产生式 (ε -productions) : $A \rightarrow \varepsilon$ (得到语言 $L - \{\varepsilon\}$)
- 消除单元产生式 (unit productions) : $A \rightarrow B$

- **定义: CFG** $G = (V, T, P, S)$, 符号 $X \in (V \cup T)$:
 - 如果 $S \xRightarrow{*} \alpha X \beta$, 称 X 是可达的(reachable)
 - 如果 $\alpha X \beta \xRightarrow{*} w (w \in T^*)$, 称 X 是产生的(generating)
 - 如果 X 同时是产生的和可达的, 即

$$S \xRightarrow{*} \alpha X \beta \xRightarrow{*} w (w \in T^*)$$
 则称 X 是有用的, 否则称 X 为无用符号。

- 计算“产生的”符号集
 - 每个 T 中的符号都是产生的
 - $A \rightarrow \alpha \in P$ 且 α 中符号都是产生的, 则 A 是产生的
- 计算“可达的”符号集
 - 符号 S 是可达的;
 - $A \rightarrow \alpha \in P$ 且 A 是可达的, 则 α 中符号都是可达的

删除全部含有“非产生的”和“非可达的”符号的产生式。

- **定理: 每个非空的CFL都能被一个不带无用符号的CFG定义。**

- **注意:**
 - 先寻找并消除全部“非产生的”符号
 - 再寻找并消除全部“非可达的”符号
 - 否则可能消除不完整
- **例: 消除如下文法无用符号:** $S \rightarrow AB \mid a, A \rightarrow b$

- 例：消除如下文法无用符号：

$$S \rightarrow AB \mid a, A \rightarrow b$$

- 消除非产生的：

- 消除非可达的：

- 例：消除如下文法无用符号：

$$S \rightarrow AB \mid a, A \rightarrow b$$

- 消除非产生的：

“产生的”符号集为 $\{a, b, S, A\}$ (a, b 是终结符； S 产生 a , A 产生 b)。 B 是非产生的

消除含有 B 的产生式, 得到 $S \rightarrow a, A \rightarrow b$ 。

- 消除非可达的：

- 例：消除如下文法无用符号：

$$S \rightarrow AB \mid a, A \rightarrow b$$

- 消除非产生的：

“产生的”符号集为 $\{a, b, S, A\}$ (a, b 是终结符； S 产生 a , A 产生 b)。 B 是非产生的

消除含有 B 的产生式, 得到 $S \rightarrow a, A \rightarrow b$ 。

- 消除非可达的：

“可达的”符号集为 $\{S, a\}$ (S 为开始符号, a 可以由 S 到达)；
消除 $\{b, A\}$, 得到 $S \rightarrow a$ 。

- 例：消除如下文法无用符号：

$$S \rightarrow AB \mid a, A \rightarrow b$$

- 消除非产生的：

“产生的”符号集为 $\{a, b, S, A\}$ (a, b 是终结符； S 产生 a , A 产生 b)。 B 是非产生的

消除含有 B 的产生式, 得到 $S \rightarrow a, A \rightarrow b$ 。

- 消除非可达的：

“可达的”符号集为 $\{S, a\}$ (S 为开始符号, a 可以由 S 到达)；
消除 $\{b, A\}$, 得到 $S \rightarrow a$ 。

如果先消除非可达的，
再消除非产生的，结果
会是怎样的呢？

- 例：消除如下文法无用符号： $S \rightarrow AB \mid a, A \rightarrow b$
 - （如果先）消除非可达的：“可达的”符号集为 $\{S, A, B, a, b\}$ ，消除 $\{\}$ （即无消除符号）
 - 消除非产生的：“产生的”符号集为 $\{a, b, S, A\}$ ，消除 B ，得到 $S \rightarrow a, A \rightarrow b$

如果先消除非可达的，最终可能会存在无用符号。

- **定义:** 文法中形如 $A \rightarrow \varepsilon$ 的产生式称为 ε -产生式。如果变元 $A \xRightarrow{*} \varepsilon$, 称 A 是可空的。
 - ε -产生式在文法定义语言时, 除产生空串外没有其他帮助。
 - 对于不包含空串的 CFL, ε -产生式不是必需的。
 - 对于 CFL L , 消除其文法中全部的 ε -产生式后, 该文法对应的语言可以表示为 $L - \{\varepsilon\}$
 - 如果空串本身不在该语言中, 则 $L = L - \{\varepsilon\}$

- 确定“可空变元”

- 如果 $A \rightarrow \varepsilon$, 则 A 是可空的;
- 如果 $B \rightarrow \alpha$ 且 α 中的每个符号都是可空的, 则 B 是可空的。

- 替换产生式: 将含有可空变元的一条产生式 $A \rightarrow X_1X_2 \cdots X_n$, 用 **一组**产生式 $A \rightarrow Y_1Y_2 \cdots Y_n$ 代替, 其中:

- 若 X_i 不是可空的, Y_i 为 X_i ;
- 若 X_i 是可空的, Y_i 为 X_i 或 ε ; (即对于每个可空变元, 它可以出现或不出现在替换后的产生式中)
- 但 Y_i 不能全为 ε 。

- 例: 消除CFG $G = (\{S, A, B\}, \{a, b\}, P, S)$ 的 ε -产生式。
 $S \rightarrow AB, A \rightarrow AaA \mid \varepsilon, B \rightarrow BbB \mid \varepsilon$

- 例: 消除CFG $G = (\{S, A, B\}, \{a, b\}, P, S)$ 的 ε -产生式。

$$S \rightarrow AB, \quad A \rightarrow AaA \mid \varepsilon, \quad B \rightarrow BbB \mid \varepsilon$$

(1) 先确定全部可空的变元:

(2) 再替换全部右部带有可空符号的产生式(注意, 每个可空变元有出现或不出现在新产生式中的两种情况):

- 例: 消除CFG $G = (\{S, A, B\}, \{a, b\}, P, S)$ 的 ε -产生式。

$$S \rightarrow AB, \quad A \rightarrow AaA \mid \varepsilon, \quad B \rightarrow BbB \mid \varepsilon$$

(1) 先确定全部可空的变元: $\{S, A, B\}$ (A, B 产生 ε ; $S \rightarrow AB$ 的右部变元只有 A, B)

(2) 再替换全部右部带有可空符号的产生式(注意, 每个可空变元有出现或不出现在新产生式中的两种情况):

- 例: 消除CFG $G = (\{S, A, B\}, \{a, b\}, P, S)$ 的 ε -产生式。

$$S \rightarrow AB, \quad A \rightarrow AaA \mid \varepsilon, \quad B \rightarrow BbB \mid \varepsilon$$

(1) 先确定全部可空的变元: $\{S, A, B\}$ (A, B 产生 ε ; $S \rightarrow AB$ 的右部变元只有 A, B)

(2) 再替换全部右部带有可空符号的产生式(注意, 每个可空变元有出现或不出现在新产生式中的两种情况):

$$S \rightarrow AB \text{ 通过替换可空符号得到 } S \rightarrow AB \mid A \mid B$$

- 例:消除CFG $G = (\{S, A, B\}, \{a, b\}, P, S)$ 的 ε -产生式。

$$S \rightarrow AB, \quad A \rightarrow AaA \mid \varepsilon, \quad B \rightarrow BbB \mid \varepsilon$$

(1) 先确定全部可空的变元: $\{S, A, B\}$ (A, B 产生 ε ; $S \rightarrow AB$ 的右部变元只有 A, B)

(2) 再替换全部右部带有可空符号的产生式(注意, 每个可空变元有出现或不出现在新产生式中的两种情况):

$$S \rightarrow AB \text{ 通过替换可空符号得到 } S \rightarrow AB \mid A \mid B$$

$$A \rightarrow AaA \text{ 通过替换可空符号得到 } A \rightarrow AaA \mid Aa \mid aA \mid a$$

- 例:消除CFG $G = (\{S, A, B\}, \{a, b\}, P, S)$ 的 ε -产生式。

$$S \rightarrow AB, \quad A \rightarrow AaA \mid \varepsilon, \quad B \rightarrow BbB \mid \varepsilon$$

(1) 先确定全部可空的变元: $\{S, A, B\}$ (A, B 产生 ε ; $S \rightarrow AB$ 的右部变元只有 A, B)

(2) 再替换全部右部带有可空符号的产生式(注意, 每个可空变元有出现或不出现在新产生式中的两种情况):

$$S \rightarrow AB \text{ 通过替换可空符号得到 } S \rightarrow AB \mid A \mid B$$

$$A \rightarrow AaA \text{ 通过替换可空符号得到 } A \rightarrow AaA \mid Aa \mid aA \mid a$$

$$B \rightarrow BbB \text{ 通过替换可空符号得到 } B \rightarrow BbB \mid Bb \mid bB \mid b$$

- 例：消除CFG $G_2 = (\{S, A, B, C\}, \{a, b\}, P, S)$ 的 ε -产生式。
 $S \rightarrow ABC, A \rightarrow aA \mid \varepsilon, B \rightarrow bB \mid \varepsilon, C \rightarrow \varepsilon$

- 例：消除CFG $G_2 = (\{S, A, B, C\}, \{a, b\}, P, S)$ 的 ε -产生式。

$$S \rightarrow ABC, A \rightarrow aA \mid \varepsilon, B \rightarrow bB \mid \varepsilon, C \rightarrow \varepsilon$$

- (1) 先确定全部可空的变元： $\{S, A, B\}$ (A, B, C 产生 ε ; $S \rightarrow ABC$ 的右部变元只有 A, B, C)

- 例：消除CFG $G_2 = (\{S, A, B, C\}, \{a, b\}, P, S)$ 的 ε -产生式。

$$S \rightarrow ABC, A \rightarrow aA \mid \varepsilon, B \rightarrow bB \mid \varepsilon, C \rightarrow \varepsilon$$

(1) 先确定全部可空的变元： $\{S, A, B\}$ (A, B, C 产生 ε ; $S \rightarrow ABC$ 的右部变元只有 A, B, C)

(2) 再替换全部右部带有可空符号的产生式(注意，每个可空变元有出现或不出现新产生式中的两种情况)：

$$S \rightarrow AB \text{ 通过替换可空符号得到 } S \rightarrow ABC \mid A \mid B \mid C \mid AB \mid AC \mid BC$$

$$A \rightarrow aA \text{ 通过替换可空符号得到 } A \rightarrow aA \mid a$$

$$B \rightarrow bB \text{ 通过替换可空符号得到 } B \rightarrow bB \mid b$$

- 例：消除CFG $G_2 = (\{S, A, B, C\}, \{a, b\}, P, S)$ 的 ε -产生式。

$$S \rightarrow ABC, A \rightarrow aA \mid \varepsilon, B \rightarrow bB \mid \varepsilon, C \rightarrow \varepsilon$$

(1) 先确定全部可空的变元： $\{S, A, B\}$ (A, B, C 产生 ε ; $S \rightarrow ABC$ 的右部变元只有 A, B, C)

(2) 再替换全部右部带有可空符号的产生式(注意，每个可空变元有出现或不出现在新产生式中的两种情况)：

$$S \rightarrow AB \text{ 通过替换可空符号得到 } S \rightarrow \cancel{A}\cancel{B}C \mid A \mid B \mid \cancel{A}B \mid \cancel{A}\cancel{C} \mid \cancel{B}\cancel{C}$$

$$A \rightarrow aA \text{ 通过替换可空符号得到 } A \rightarrow aA \mid a$$

$$B \rightarrow bB \text{ 通过替换可空符号得到 } B \rightarrow bB \mid b$$

C 是无用符号

- **确定单元对：**如果有 $A \xRightarrow{*} B$ ，则称 $[A, B]$ 为单元对。
 - (1) $A \rightarrow B \in P$ ，则 $[A, B]$ 是单元对；
 - (2) 若 $[A, B]$ 和 $[B, C]$ 都是单元对，则 $[A, C]$ 是单元对
- **消除单元产生式**
 - 删除全部形为 $A \rightarrow B$ 的单元产生式；
 - 对每个单元对 $[A, B]$ ，将 B 的产生式复制给 A 。

- 定理: 每个不带 ε 的CFL都可由一个不带无用符号, ε -产生式和单元产生式的文法来定义。

- 例: 给定CFG $G = (\{S, A, B\}, \{0, 1\}, P, S)$, 其中 P 为 $S \rightarrow A \mid B \mid 0S1$, $A \rightarrow 0A \mid 0$, $B \rightarrow 1B \mid 1$ 。消除文法的单元产生式。

- 例: 给定CFG $G = (\{S, A, B\}, \{0, 1\}, P, S)$, 其中 P 为 $S \rightarrow A \mid B \mid 0S1$, $A \rightarrow 0A \mid 0$, $B \rightarrow 1B \mid 1$ 。消除文法的单元产生式。

1. 先确定单元产生式: $S \rightarrow A$, $S \rightarrow B$, 即单元对为 $[S, A], [S, B]$

2. 删除 $S \rightarrow A, S \rightarrow B$, 并将 A 的产生式赋给 S, B 的产生式赋给 S , 有

$$S \rightarrow 0A \mid 0, \quad S \rightarrow 1B \mid 1$$

3. 最终得到:

$$S \rightarrow 0A \mid 0, \quad S \rightarrow 1B \mid 1, \quad S \rightarrow 0S1$$
$$A \rightarrow 0A \mid 0, \quad B \rightarrow 1B \mid 1$$

- **文法简化步骤的顺序是重要的，一个可靠的顺序是**
 - 消除 ε – 产生式
 - 消除单元产生式
 - 消除非产生的无用符号
 - 消除非可达的无用符号

乔姆斯基范式和格雷巴赫范式

- 文法的格式多种多样，是否存在规范化的表示形式？
- 乔姆斯基范式 (Chomsky Normal Form, CNF) 和格雷巴赫范式 (Greibach Normal Form, GNF) 对文法的格式分别进行了规范化。

- 定理：每个不带 ε 的CFL都可以由这样的CFG G 定义, G 中每个产生式的形式都为

$$A \rightarrow BC \text{ 或 } A \rightarrow a$$

这里的 A, B 和 C 是变元, a 是终结符。

- 设文法不带无用符号， ε -产生式和单元产生式（任何文法都可以化成这种形式）。
- 考虑文法每个形式为 $A \rightarrow X_1X_2 \cdots X_m$ ($m \geq 2$)的产生式，若 X_i 为终结符 a ，则引进新变元 C_a 替换 X_i 并增加新产生式 $C_a \rightarrow a$ 。
- 所有产生式的形式变为 $A \rightarrow B_1B_2 \cdots B_m$ ($m \geq 2$)和 B_i (即 C_a) $\rightarrow a$ 。
- 对于 $A \rightarrow B_1B_2 \cdots B_m$ ($m > 2$)，引入新变元 $D_1, D_2, \cdots, D_{m-2}$ ，将其替换为一组级联产生式

$$A \rightarrow B_1D_1, D_1 \rightarrow B_2D_2, \cdots D_{m-2} \rightarrow B_{m-1}B_m$$

即先把 $B_1B_2 \cdots B_m$ 划分为两部分： B_1 和 $D_1: B_2 \cdots B_m$ ；然后 D_1 又可以划分为两部分 B_2 和 $D_2: B_3 \cdots B_m$ ，依次类推。

- 例: CFG $G = (\{S, A, B\}, \{a, b\}, P, S)$, 产生式集合 P 为:
$$S \rightarrow bA | aB, A \rightarrow bAA | aS | a, B \rightarrow aBB | bS | b$$

设计等价的CNF文法。

- 例: CFG $G = (\{S, A, B\}, \{a, b\}, P, S)$, 产生式集合 P 为:
 $S \rightarrow bA | aB, A \rightarrow bAA | aS | a, B \rightarrow aBB | bS | b$

设计等价的CNF文法。

1. 引进新变元替换每个产生式中的终结符。比如, 对于 a 引进新变元 C_a 替换各个产生式中的 a , 并增加新产生式 $C_a \rightarrow a$ 。

2. 所有产生式的形式变为 $A \rightarrow B_1 B_2 \cdots B_m (m \geq 2)$ 和 B_i (即 C_a) $\rightarrow a$ 。
 对于 $A \rightarrow B_1 B_2 \cdots B_m (m \geq 2)$, 引入新变元 $D_1, D_2, \dots, D_{m-2}$, 将其替换为一组级联产生式

$$A \rightarrow B_1 D_1, D_1 \rightarrow B_2 D_2, \dots, \\ D_{m-2} \rightarrow B_{m-1} B_m$$

- 例: CFG $G = (\{S, A, B\}, \{a, b\}, P, S)$, 产生式集合 P 为:
 $S \rightarrow bA | aB, A \rightarrow bAA | aS | a, B \rightarrow aBB | bS | b$

设计等价的CNF文法。

1. 增加新变元替换终结符:

$$C_b \rightarrow b, C_a \rightarrow a$$

$$S \rightarrow bA | aB \rightarrow S \rightarrow C_b A | C_a B$$

$$A \rightarrow bAA | aS | a \rightarrow A \rightarrow C_b AA | C_a S | a$$

$$B \rightarrow aBB | bS | b \rightarrow B \rightarrow C_a BB | C_b S | b$$

2. 对仅包含变元、长度大于2的产生式进行转换:

$$A \rightarrow C_b AA \rightarrow A \rightarrow C_b D_1, D_1 \rightarrow AA$$

$$B \rightarrow C_a BB \rightarrow B \rightarrow C_a D_2, D_2 \rightarrow BB$$

- 例: CFG $G = (\{S, A, B\}, \{a, b\}, P, S)$, 产生式集合 P 为:

$$S \rightarrow bA | aB, A \rightarrow bAA | aS | a, B \rightarrow aBB | bS | b$$

设计等价的CNF文法。

1. 增加新变元替换终结符:

$$C_b \rightarrow b, C_a \rightarrow a$$

$$S \rightarrow bA | aB \rightarrow S \rightarrow C_b A | C_a B$$

$$A \rightarrow bAA | aS | a \rightarrow A \rightarrow C_b AA | C_a S | a$$

$$B \rightarrow aBB | bS | b \rightarrow B \rightarrow C_a BB | C_b S | b$$

2. 对仅包含变元、长度大于2的产生式进行转换:

$$A \rightarrow C_b AA \rightarrow A \rightarrow C_b D_1, D_1 \rightarrow AA$$

$$B \rightarrow C_a BB \rightarrow B \rightarrow C_a D_2, D_2 \rightarrow BB$$

注意: 不能因为原来有产生式 $A \rightarrow a$ 和 $B \rightarrow b$ 而放弃引入变量 C_a 、 C_b 和产生式 $C_a \rightarrow a$ 、 $C_b \rightarrow b$

- 例: CFG $G = (\{S, A, B\}, \{a, b\}, P, S)$, 产生式集合 P 为:

$$S \rightarrow bA | aB, A \rightarrow bAA | aS | a, B \rightarrow aBB | bS | b$$

等价的CNF结果:

$$S \rightarrow C_b A | C_a B$$

$$A \rightarrow C_b D_1 | C_a S | a$$

$$B \rightarrow C_a D_2 | C_b S | b$$

$$D_1 \rightarrow AA, D_2 \rightarrow BB$$

$$C_b \rightarrow b, C_a \rightarrow a$$

- 例：将下面的CFG G转为等价的CNF。

G:

$S \Rightarrow AS \mid BABC$

$A \Rightarrow A1 \mid 0A1 \mid 01$

$B \Rightarrow 0B \mid 0$

$C \Rightarrow 1C \mid 1$

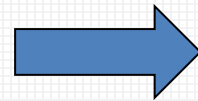
- 例：将下面的CFG G转为等价的CNF。

G:

$$S \Rightarrow AS \mid BABC$$

$$A \Rightarrow A1 \mid 0A1 \mid 01$$

$$B \Rightarrow 0B \mid 0$$

$$C \Rightarrow 1C \mid 1$$


G in CNF:

$$X_0 \Rightarrow 0$$

$$X_1 \Rightarrow 1$$

$$S \Rightarrow AS \mid BY_1$$

$$Y_1 \Rightarrow AY_2$$

$$Y_2 \Rightarrow BC$$

$$A \Rightarrow AX_1 \mid X_0Y_3 \mid X_0X_1$$

$$Y_3 \Rightarrow AX_1$$

$$B \Rightarrow X_0B \mid 0$$

$$C \Rightarrow X_1C \mid 1$$

- 定理: 每个不带 ε 的CFL都可以由这样的CFG G 定义, G 中每个产生式的形式都为

$$A \rightarrow a\alpha$$

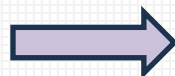
其中 A 是变元, a 是终结符, α 是零或多个变元的串。

- GNF (Greibach Normal Form) 中每个产生式都会引入一个终结符。长度为 n 的串的派生恰好是 n 步

$$S \rightarrow XB \mid AA$$

$$A \rightarrow a \mid SA$$

$$B \rightarrow b$$

$$X \rightarrow a$$


$$S \rightarrow aB \mid aCA \mid aBACA \mid aA \mid aBAA$$

$$A \rightarrow aC \mid aBAC \mid a \mid aBA$$

$$C \rightarrow aCAC \mid aBACAC \mid aAC \mid aBAAC$$

$$C \rightarrow aCA \mid aBACA \mid aA \mid aBAA$$

$$B \rightarrow b$$

$$X \rightarrow a$$

- **CFG，派生，最左派生与最右派生，句型，句子**
- **语法分析树，派生与语法树的等价性，二义性文法与文法的固有二义性**
- **CFG的化简**
 - 无用符号、 ε – 产生式和单元产生式的消除
- **CFG的范式**
 - CNF和GNF